

Used Car Price Prediction

Presented by:

Ziad Elmorsy 202303005

Maha mikkawi 202300955

Mostafa Elzeiny 202302013

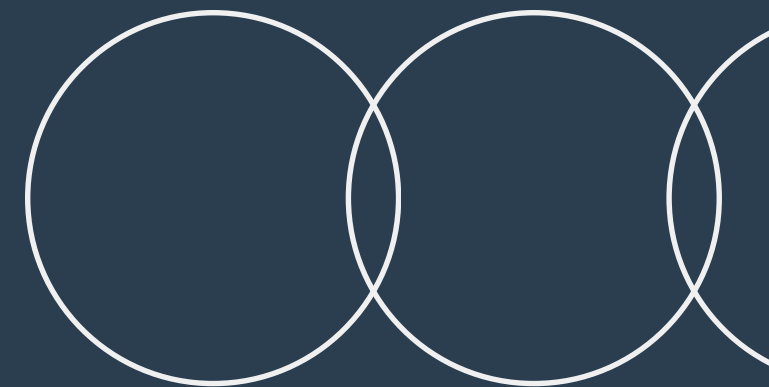
Gamal Mohamed 202302947



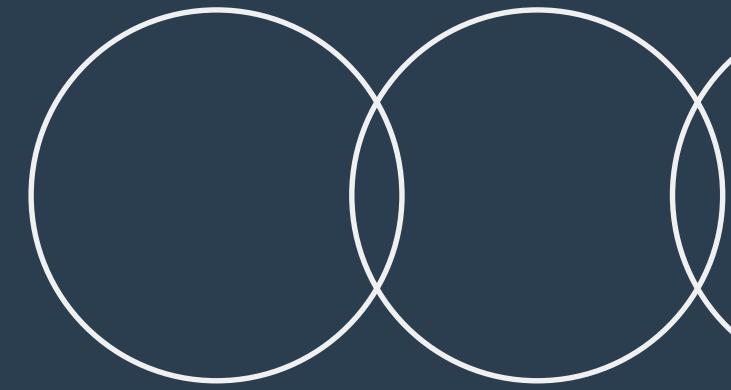
Problem Statement

Used car prices depend on many factors such as age, mileage, brand, and fuel type, which makes pricing difficult.

This project builds a machine learning model to predict used car prices using vehicle features.



Dataset Overview



Source:

Kaggle – Used Cars Price Prediction

Target Variable:

Price

```
display(df.head())
```

	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage	Engine	Power	Seats	Price
0	Maruti Wagon R LXI CNG	Mumbai	2010	72000	CNG	Manual	First	26.6 km/kg	998 CC	58.16 bhp	5.0	1.75
1	Hyundai Creta 1.6 CRDi SX Option	Pune	2015	41000	Diesel	Manual	First	19.67 kmpl	1582 CC	126.2 bhp	5.0	12.50
2	Honda Jazz V	Chennai	2011	46000	Petrol	Manual	First	18.2 kmpl	1199 CC	88.7 bhp	5.0	4.50
3	Maruti Ertiga VDI	Chennai	2012	87000	Diesel	Manual	First	20.77 kmpl	1248 CC	88.76 bhp	7.0	6.00
4	Audi A4 New 2.0 TDI Multitronic	Coimbatore	2013	40670	Diesel	Automatic	Second	15.2 kmpl	1968 CC	140.8 bhp	5.0	17.74

Data Cleaning

The data preprocessing steps involved several critical actions to ensure model accuracy:

- **Removing duplicates:** All duplicate entries were eliminated to maintain data integrity.
- **Handling missing values:** Missing values were addressed by imputing with mean or median where applicable.

3 Data Cleaning

```
#3) Remove common index columns
index_like_cols = [c for c in df.columns if c.lower().startswith("unnamed")]
if index_like_cols:
    df.drop(columns=index_like_cols, inplace=True)

# 1) Remove duplicate rows
df = df.drop_duplicates()

# 2) Drop rows with missing target value (Price)
df = df.dropna(subset=["Price"])

# 3) Handle missing values in feature columns
feature_cols = [c for c in df.columns if c != "Price"]

# Identify numerical and categorical features
numeric_cols = df[feature_cols].select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = [c for c in feature_cols if c not in numeric_cols]

# Fill missing numerical values with median
for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())

# Fill missing categorical values with mode
for col in categorical_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
```

Data Preprocessing

Feature Engineering & Transformation

- Converted mixed-type string features into numerical values
- (e.g., engine capacity, mileage, power)
- Encoded categorical variables using label encoding
 - Maintains a compact feature space
 - Efficient for tree-based models
- Detected and treated outliers using the Interquartile Range (IQR) method
 - Extreme values were capped to reduce skewness
- Applied feature scaling where required
 - Ensures numerical stability across models
- Prepared the final feature matrix for model training and evaluation

```
# Outlier handling

def iqr_cap(df_in: pd.DataFrame, cols: list[str], k: float = 1.5) -> pd.DataFrame:
    df_out = df_in.copy()
    for col in cols:
        q1 = df_out[col].quantile(0.25)
        q3 = df_out[col].quantile(0.75)
        iqr = q3 - q1
        lower = q1 - k * iqr
        upper = q3 + k * iqr
        df_out[col] = df_out[col].clip(lower, upper)
    return df_out

# Choose numeric features to cap (exclude target by default)
numeric_to_cap = [c for c in numeric_cols if c != target_col]

df = iqr_cap(df, numeric_to_cap, k=1.5)

print("Outlier capping done.")
```

Outlier capping done.

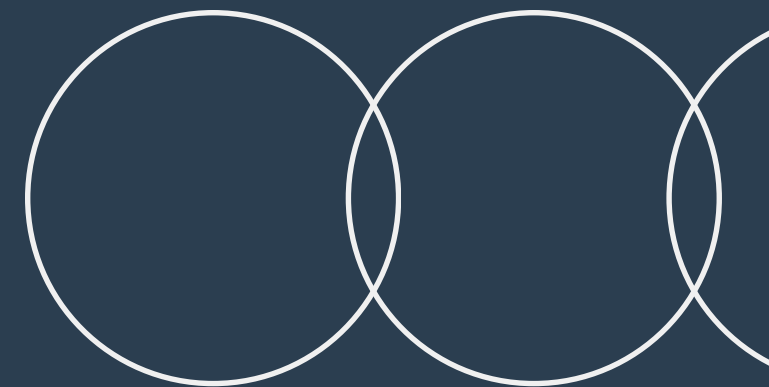
Train-Test Split Strategy

Hold-Out Validation Explained

The hold-out validation approach involves splitting the dataset into training and testing sets, allowing for objective evaluation of model performance on unseen data while minimizing overfitting risks.

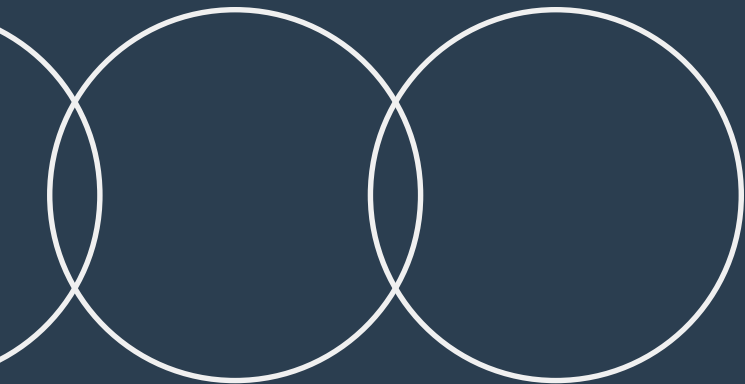
```
# 1) Split features and target
X = df.drop(columns=["Price"])
y = df["Price"]

# 2) Train-Test Split (Hold-out validation)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```



Baseline Models

Evaluating Linear Regression, Decision Tree, Random Forest, Polynomial Regression and Gradient Boosting



Linear Regression

Predicts price using a simple linear relationship between features and the target

Decision Tree

Makes predictions by learning decision rules from the data

Random Forest

Combines multiple decision trees to improve prediction accuracy

Polynomial Regression

Models non-linear relationships by adding polynomial features

Gradient Boosting

Builds models sequentially to correct previous prediction errors

Trial 1 — Baseline Models

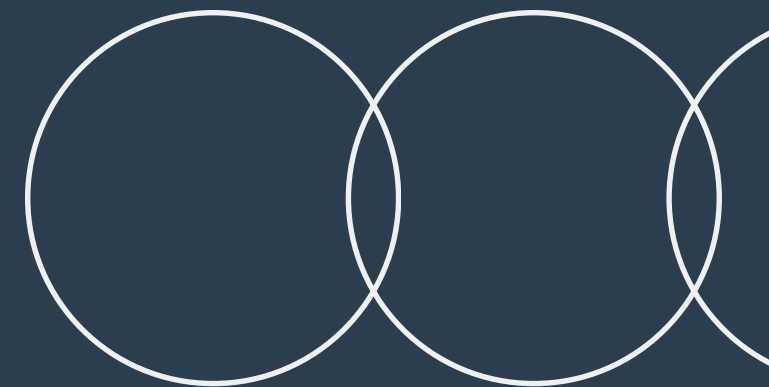
Establish a baseline without optimization or feature engineering

Models:

- Linear Regression
- Decision Tree Regressor
- Random Forest Regressor
- Polynomial Regression
- Gradient Boosting Regressor

Evaluation Metrics

- RMSE (Root Mean Squared Error)
- MAE (Mean Absolute Error)
- R^2 Score
- Train vs Test performance gap



Trial 1 — Results

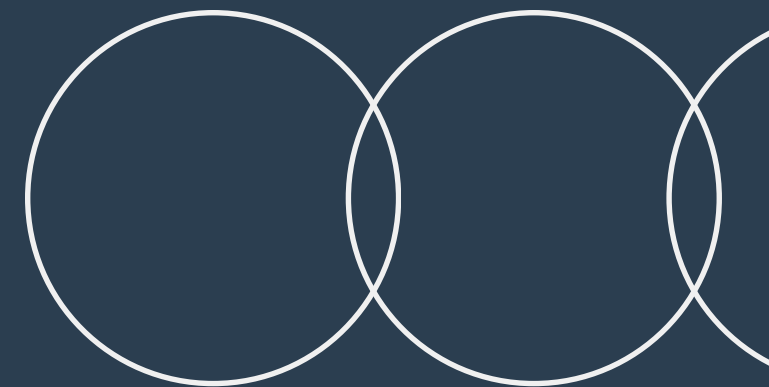
Results Summary:

	Model	Train_RMSE	Test_RMSE	Train_MAE	Test_MAE	Train_R2	Test_R2	Gap_RMSE	Gap_R2
2	Random Forest	1.348039	4.121175	0.546888	1.952979	0.985539	0.861985	2.773137	0.123555
3	Gradient Boosting	2.777663	4.194986	1.494925	2.125280	0.938604	0.856997	1.417323	0.081608
1	Decision Tree	0.018881	5.815393	0.000891	2.546964	0.999997	0.725183	5.796512	0.274814
5	Ridge (optional)	6.293137	6.257231	3.689830	3.855873	0.684853	0.681837	-0.035906	0.003016
0	Linear Regression	6.293136	6.257393	3.690053	3.856215	0.684853	0.681821	-0.035743	0.003032
4	Polynomial Regression	4.231028	6.892020	2.310672	2.950125	0.857547	0.614008	2.660992	0.243539

Trial 2: Feature Engineering & Optimization

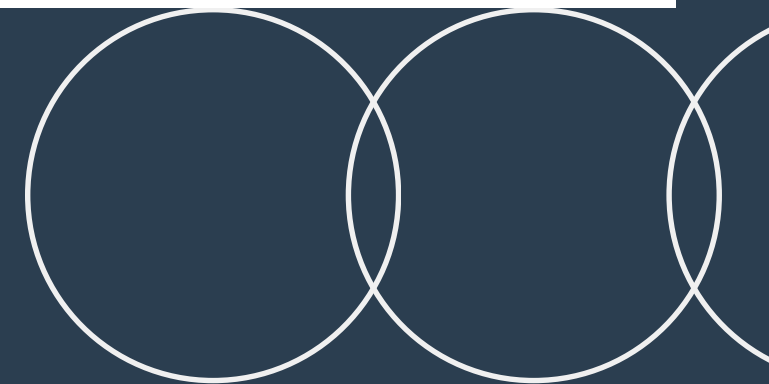
What Was Done

- Applied hyperparameter tuning to all candidate models
- Used cross-validation to ensure robust performance comparison
- Optimized models based on test RMSE
- Selected the best-performing configuration for each model



Trial 2 — Results

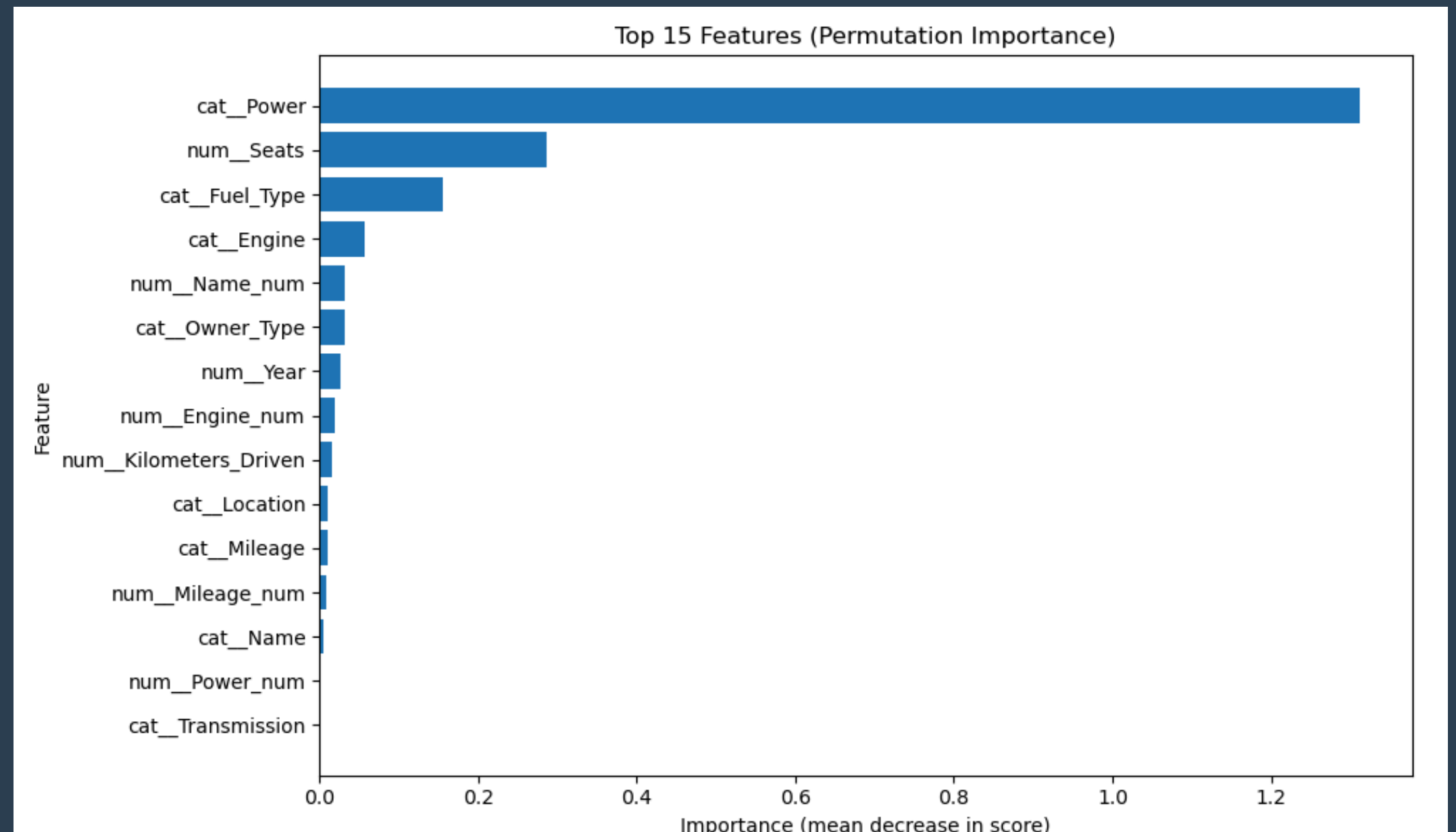
	Model	Train_RMSE	Test_RMSE	Train_MAE	Test_MAE	Train_R2	Test_R2	Gap_RMSE	Gap_R2
2	Gradient Boosting (Tuned)	1.422214	3.852841	0.913209	1.789032	0.983904	0.879372	2.430626	0.104532
1	Random Forest (Tuned)	1.307177	3.889424	0.531027	1.901928	0.986403	0.877071	2.582247	0.109332
0	Decision Tree (Tuned)	3.075092	5.192446	1.371230	2.300434	0.924752	0.780907	2.117354	0.143845
4	Ridge (optional) (Tuned)	6.297162	6.247184	3.670604	3.827125	0.684450	0.682858	-0.049978	0.001592
3	Polynomial Regression (Tuned)	4.231028	6.892020	2.310672	2.950125	0.857547	0.614008	2.660992	0.243539



Feature Importance

Feature Importance

- Computed using permutation importance
- Based on the best tuned model
- Highlights the most influential predictors



Feature Importance Analysis

Understanding the Role of Car Power

The feature importance analysis indicates that **Car Power significantly influences** used car prices. Understanding this relationship can enhance our pricing model and improve accuracy in predictions.



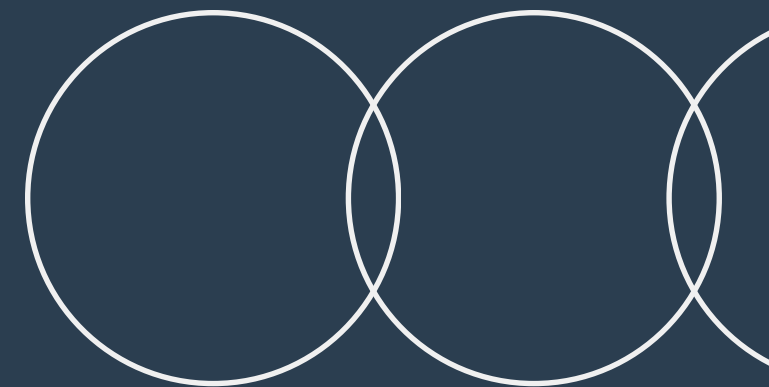
Final Model Selection

Chosen model:

Gradient Boosting (tuned)

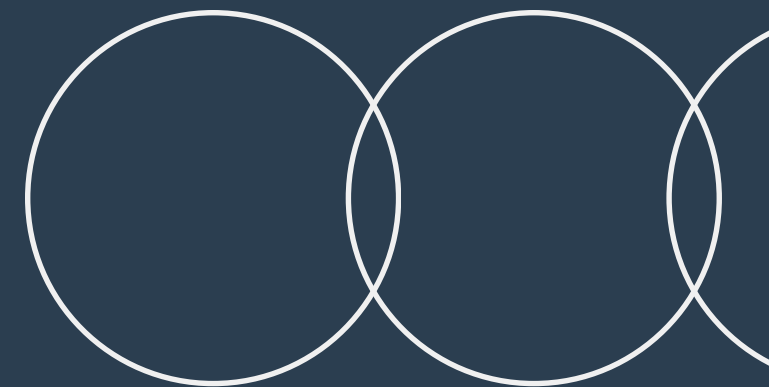
Why?

- Achieved the lowest test RMSE among all models
- Demonstrated strong generalization performance
- Maintained a balanced train–test error gap
- Showed stable behavior across cross-validation



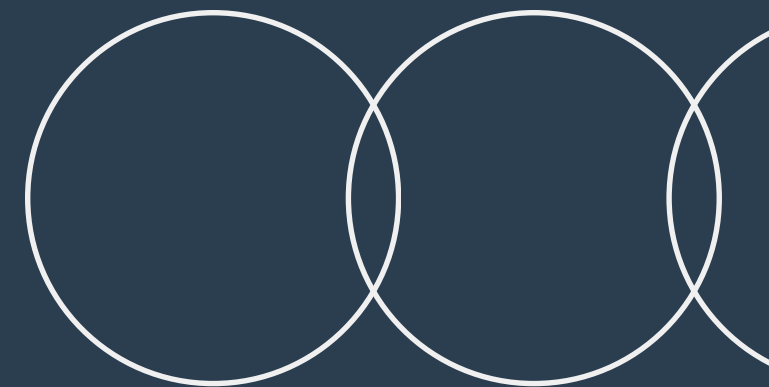
Key Learnings

- Data preprocessing has a strong impact on model performance
- Feature engineering helps models learn more effectively
- Outlier handling improves prediction stability
- Model complexity affects overfitting and generalization
- Hyperparameter tuning improves overall accuracy
- Feature importance helps explain model behavior



Conclusion

- A complete machine learning pipeline was developed for price prediction
- Careful data preprocessing improved model performance and stability
- Multiple models were evaluated and compared fairly
- Hyperparameter tuning enhanced generalization
- Gradient Boosting achieved the best overall performance
- Feature importance provided interpretability and confidence in the results



Any questions or comments?

We appreciate your attention and feedback today!

